

ĐỒ ÁN 1

HỌC CÓ GIÁM SÁT VÀ ỨNG DỤNG

Môn học: Nhập môn học máy
Mã môn: CSC14005
Học kỳ: HỌC KỲ 2 – 2025/2026

Thông tin giảng viên

GV Thực hành: Th.S Lê Nhật Nam
E-mail: lnnam@fit.hcmus.edu.vn
Bộ môn: Khoa học máy tính

Mục lục

1	Giới thiệu Tổng quan	3
1.1	Tài liệu tham khảo chính	3
1.2	Cấu trúc Đồ án	3
1.3	Quy định chung	4
2	Phần 1 – Bài toán Hồi quy (Regression)	4
2.1	Cơ sở Lý thuyết	4
2.1.1	Bài toán Hồi quy	5
2.1.2	Các hàm cơ sở thường gặp	5
2.1.3	Ước lượng tham số: Bình phương tối thiểu (OLS)	5
2.1.4	Regularization – Chính quy hoá	6
2.1.5	Tiếp cận xác suất: Maximum Likelihood và Maximum a posteriori	6
2.1.6	Phân tích Bias–Variance	6
2.1.7	Hồi quy Bayesian [Nâng cao]	6
2.1.8	Các biến thể Gradient Descent	7
2.1.9	Kernel Ridge Regression [Nâng cao]	7
2.1.10	Gaussian Process Regression [Nâng cao]	8
2.2	Yêu cầu Thực hiện – Phần 1	8
2.2.1	Chọn bộ dữ liệu	8
2.2.2	Phân tích và Tiền xử lý Dữ liệu	8
2.2.3	Xây dựng và Huấn luyện Mô hình	9
2.2.4	Đánh giá Mô hình	10
2.2.5	Phân tích và Thảo luận	11
2.3	Phân bố Điểm – Phần 1	11
3	Phần 2 – Bài toán Phân lớp (Classification)	12
3.1	Cơ sở Lý thuyết	12
3.1.1	Bài toán Phân lớp	12
3.1.2	Các chiến lược phân lớp	12
3.1.3	Hàm phân quyết tuyến tính	12
3.1.4	Phân tích phân tách tuyến tính Fisher (Fisher’s LDA)	13

3.1.5	Hồi quy Logistic (Logistic Regression)	13
3.1.6	Mô hình sinh Gaussian (Generative LDA)	13
3.1.7	Xấp xỉ Laplace [Nâng cao]	14
3.1.8	Hàm Probit [Nâng cao]	14
3.1.9	Thuật toán Newton–Raphson / IRLS [Nâng cao]	14
3.1.10	Thuật toán Perceptron	14
3.1.11	Kernel Logistic Regression [Nâng cao]	15
3.1.12	Thuật toán tối ưu hóa	15
3.2	Yêu cầu Thực hiện – Phần 2	15
3.2.1	Chọn bộ dữ liệu	15
3.2.2	Xây dựng và Huấn luyện Mô hình	16
3.2.3	Đánh giá Mô hình	17
3.2.4	Phân tích và Thảo luận	18
3.3	Phân bố Điểm – Phần 2	18
4	Phần 3 – Phân tích So sánh và Kỹ năng Nghiên cứu	19
4.1	Kết nối Hồi quy và Phân lớp	19
4.2	Bảng so sánh toàn diện	19
4.3	Phân tích mức độ nhạy cảm (sensitivity) và tính bền vững (robustness)	20
4.4	Khả năng tái hiện lại thí nghiệm	20
5	Tổng kết Điểm số	20
6	Quy định Nộp bài	21
6.1	Cấu trúc thư mục nộp	21
6.2	Yêu cầu về Code	22
6.3	Yêu cầu về Báo cáo	22
6.4	Hình thức và Hạn nộp	22

1. Giới thiệu Tổng quan

Đồ án này yêu cầu sinh viên tìm hiểu và áp dụng các thuật toán **học máy có giám sát** (supervised learning) để giải quyết hai bài toán cơ bản: **hồi quy** (regression) và **phân lớp** (classification). Sinh viên cần trang bị nền tảng lý thuyết vững chắc, đồng thời thể hiện khả năng vận dụng vào dữ liệu thực tế thông qua các thí nghiệm có hệ thống và đánh giá kết quả rõ ràng.

Mục tiêu của đồ án

Sau khi hoàn thành đồ án, sinh viên có khả năng:

- Hiểu và trình bày được nền tảng toán học của các mô hình tuyến tính trong học máy có giám sát.
- Cài đặt và áp dụng các thuật toán hồi quy tuyến tính lên bộ dữ liệu thực tế.
- Cài đặt và áp dụng các thuật toán phân lớp tuyến tính lên bộ dữ liệu thực tế.
- Đánh giá hiệu năng mô hình bằng các chỉ số phù hợp và phân tích kết quả một cách có phê phán.
- Trình bày báo cáo khoa học rõ ràng, mạch lạc, có trích dẫn tài liệu tham khảo.

1.1. Tài liệu tham khảo chính

Sách tham khảo

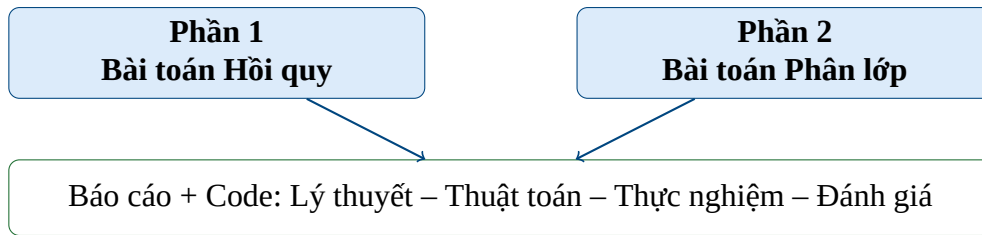
Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

- **Phần 1 – Hồi quy:** Chương 3: *Linear Models for Regression*
- **Phần 2 – Phân lớp:** Chương 4: *Linear Models for Classification*

Ngoài ra, sinh viên có thể tham khảo thêm các bài báo, giáo trình liên quan đến học máy có giám sát để làm phong phú thêm cho báo cáo đồ án.

1.2. Cấu trúc Đồ án

Đồ án gồm hai phần chính như sau:



1.3. Quy định chung

! Lưu ý

- Mỗi nhóm gồm **tối đa 5 sinh viên**.
- **Không sử dụng dữ liệu tổng hợp (synthetic data)**. Tất cả thí nghiệm phải thực hiện trên **dữ liệu thực tế** lấy từ các nguồn uy tín (UCI ML Repository, Kaggle, v.v.) hoặc **dữ liệu tự thu thập** được từ các trang web, kho dữ liệu được công khai (nếu sinh viên tự thu thập, tham khảo quy trình Nhập môn Khoa học Dữ liệu).
- Ngôn ngữ lập: **Python 3.x**. Sử dụng các thư viện: `numpy`, `pandas`, `matplotlib`, `scikit-learn`, `seaborn`, `plotly`.
- Báo cáo nên được viết bằng $\text{\LaTeX}$, trình bày sạch đẹp, có đánh số công thức, bảng biểu và hình vẽ rõ ràng. Các báo cáo trình bày thiếu mạch lạc, hình vẽ và đồ thị không đảm bảo chất lượng đều sẽ bị trừ điểm.
- Nguồn trích dẫn tài liệu phải được ghi rõ ràng theo chuẩn APA hoặc IEEE. Hạn chế trích dẫn các tài liệu không chính thống như Wikipedia.
- Tổng điểm cơ bản: **100 điểm**. Có thêm **10 điểm bonus** cho các yêu cầu nâng cao.

2. Phần 1 – Bài toán Hồi quy (Regression)

Tổng quan Phần 1

Sinh viên tìm hiểu và áp dụng các **mô hình tuyến tính cho bài toán hồi quy** (Linear Models for Regression). Sinh viên **nên** tham khảo **Chương 3** của sách *Pattern Recognition and Machine Learning* (Bishop, 2006).

Tổng điểm Phần 1: **[50 điểm]**

2.1. Cơ sở Lý thuyết

2.1.1. Bài toán Hồi quy

Bài toán hồi quy đặt ra như sau: cho một tập dữ liệu huấn luyện $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N$, trong đó $\mathbf{x}_n \in \mathbb{R}^D$ là vector đầu vào và $t_n \in \mathbb{R}$ là giá trị đầu ra (biến mục tiêu), hãy xây dựng một hàm $y(\mathbf{x})$ sao cho:

$$t \approx y(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \beta^{-1}), \quad (1)$$

trong đó ϵ là nhiễu Gaussian với độ chính xác (precision) β .

Định nghĩa 2.1 (Mô hình tuyến tính cơ sở). Mô hình hồi quy tuyến tính được định nghĩa như sau:

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}), \quad (2)$$

trong đó $\boldsymbol{\phi}(\mathbf{x}) = [\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x})]^\top$ là vector các hàm cơ sở (basis functions), $\phi_0(\mathbf{x}) = 1$, và $\mathbf{w} = [w_0, w_1, \dots, w_{M-1}]^\top$ là vector trọng số.

2.1.2. Các hàm cơ sở thường gặp

Sinh viên cần tìm hiểu và so sánh ít nhất **ba loại hàm cơ sở** sau:

Loại hàm cơ sở	Công thức	Đặc điểm
Tuyến tính	$\phi_j(x) = x_j$	Simple
Đa thức	$\phi_j(x) = x^j$	Approximate non-linear locally
Gaussian RBF	$\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$	localized influence, infinite differentiability, strict positive-definiteness
Sigmoid	$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$	Logistic variations

2.1.3. Ước lượng tham số: Bình phương tối thiểu (OLS)

Hàm mất mát (sum-of-squares error) được định nghĩa như sau:

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}_n)\}^2 = \frac{1}{2} \|\mathbf{t} - \boldsymbol{\Phi} \mathbf{w}\|^2, \quad (3)$$

trong đó $\boldsymbol{\Phi} \in \mathbb{R}^{N \times M}$ là ma trận thiết kế (design matrix) với $\Phi_{nj} = \phi_j(\mathbf{x}_n)$. Tùy vào việc định nghĩa hàm $\phi_j(\cdot)$ mà chúng ta có những kết quả khác nhau. Sinh viên có thể xem nó giống như phương pháp Kernel. Tham khảo: [Machine Learning with Kernel Methods, PSL University](#)

Định lý 2.1 (Nghiem bình phương tối thiểu). *Nghiem tối ưu của $E_D(\mathbf{w})$ là:*

$$\mathbf{w}^* = \boldsymbol{\Phi}^\dagger \mathbf{t} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{t}, \quad (4)$$

trong đó $\boldsymbol{\Phi}^\dagger$ là giả nghịch đảo Moore–Penrose của $\boldsymbol{\Phi}$. Nghiệm này có tên gọi là *Normal Equations*.

Tham khảo một số chứng minh: [Tham khảo 1](#), [Tham khảo 2](#)

2.1.4. Regularization – Chính quy hoá

Để giảm thiểu hiện tượng *overfitting*, người ta thêm số hạng phạt vào hàm mất mát. Phương pháp phổ biến là **Ridge Regression** (chuẩn hóa L2):

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2. \quad (5)$$

Nghiệm tối ưu tương ứng:

$$\mathbf{w}_{\text{ridge}}^* = (\Phi^\top \Phi + \lambda \mathbf{I})^{-1} \Phi^\top \mathbf{t}. \quad (6)$$

Ngoài ra, sinh viên cần tìm hiểu **Lasso Regression** (chuẩn hóa L1):

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1. \quad (7)$$

Bên cạnh đó, có một số phương pháp đặt (điều) chỉnh khác thường gặp như: Elastic (L1 + L2), Dropout, Data Augmentation, và Early stopping.

2.1.5. Tiếp cận xác suất: Maximum Likelihood và Maximum a posteriori

Với giả thiết $p(\mathbf{t} | \mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(\mathbf{t} | y(\mathbf{x}, \mathbf{w}), \beta^{-1})$, hàm log-likelihood được định nghĩa như sau:

$$\ln p(\mathbf{t} | \mathbf{w}, \beta) = \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \beta E_D(\mathbf{w}). \quad (8)$$

Khi ta cực đại log-likelihood thì cũng tương đương với việc tối thiểu hàm $E_D(\mathbf{w})$, tức là *Maximum Likelihood (MLE)* hay **Triển vọng cực đại** cho nghiệm giống như phương pháp Bình Phương Tối Thiểu (OLS).

Bên cạnh đó, hướng tiếp cận **Maximum a posteriori (MAP)** hay **Cực đại xác suất hậu nghiệm** đặt prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \alpha^{-1} \mathbf{I})$, khi đó nghiệm MAP tương đương với Ridge Regression với $\lambda = \alpha/\beta$.

2.1.6. Phân tích Bias–Variance

Sai số kỳ vọng của mô hình trên điểm kiểm tra $\mathbf{x}^*$ có thể phân tách thành:

$$\mathbb{E}[(t - y(\mathbf{x}^*))^2] = \underbrace{(\mathbb{E}[y(\mathbf{x}^*)] - h(\mathbf{x}^*))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(y(\mathbf{x}^*) - \mathbb{E}[y(\mathbf{x}^*)])^2]}_{\text{variance}} + \underbrace{\sigma_\epsilon^2}_{\text{noise}}, \quad (9)$$

trong đó $h(\mathbf{x}^*)$ là hàm mục tiêu giá trị thực. Khi tăng độ phức tạp mô hình làm giảm bias nhưng lại tăng variance – đây là **đánh đổi bias–variance** (bias–variance tradeoff). Các kỹ thuật regularization cho phép chúng ta kiểm soát sự đánh đổi này thông qua siêu tham số λ .

2.1.7. Hồi quy Bayesian [Nâng cao]

Hồi quy Bayesian tính phân phối hậu nghiệm (posterior) của tham số:

$$p(\mathbf{w} | \mathbf{t}) = \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N), \quad (10)$$

trong đó:

$$\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^\top \Phi, \quad \mathbf{m}_N = \beta \mathbf{S}_N \Phi^\top \mathbf{t}. \quad (11)$$

Phân phối dự đoán (predictive distribution):

$$p(t^* | \mathbf{x}^*, \mathbf{t}) = \mathcal{N}(t^* | \mathbf{m}_N^\top \phi(\mathbf{x}^*), \sigma_N^2(\mathbf{x}^*)), \quad (12)$$

trong đó $\sigma_N^2(\mathbf{x}^*) = \beta^{-1} + \phi(\mathbf{x}^*)^\top \mathbf{S}_N \phi(\mathbf{x}^*)$.

2.1.8. Các biến thể Gradient Descent

Normal Equations có độ phức tạp $\mathcal{O}(NM^2 + M^3)$ – không khả thi khi N hoặc M lớn. Các phương pháp tối ưu hóa lặp là giải pháp thay thế. Sinh viên có thể tham khảo các thuật toán như sau:

Thuật toán	Cập nhật $\mathbf{w}$	Chi phí/bước	Đặc điểm
Batch GD	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$	$\mathcal{O}(NM)$	Ổn định, chậm
SGD	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n$	$\mathcal{O}(M)$	Nhanh
Mini-batch	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_B$	$\mathcal{O}(BM)$	Cân bằng
Momentum	$\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla E; \mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$	$\mathcal{O}(NM)$	Giảm dao động
Adam	Adaptive per-parameter learning rate	$\mathcal{O}(NM)$	Thích nghi cao

Thông thường, chúng ta hay sử dụng phương pháp hướng giảm (descent method) để tối ưu. Tùy vào thông tin cục bộ mà thuật toán sử dụng mà ta có thể phân loại nó thành hai nhóm chính: các thuật toán bậc nhất (first-order) và bậc hai (second-order). Tham khảo sách: [Algorithm for Optimization](#) của Mykel J. Kochenderfer và Tim A. Wheeler.

2.1.9. Kernel Ridge Regression [Nâng cao]

Bằng cách áp dụng **kernel trick**, ta có thể mở rộng Ridge Regression sang không gian đặc trưng vô hạn chiều mà không cần tính tường minh $\phi(\mathbf{x})$:

$$\mathbf{w}^* = \Phi^\top (\Phi \Phi^\top + \lambda \mathbf{I}_N)^{-1} \mathbf{t}. \quad (13)$$

Dự đoán tại điểm mới:

$$y(\mathbf{x}^*) = \mathbf{k}(\mathbf{x}^*)^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t}, \quad (14)$$

trong đó $K_{nm} = k(\mathbf{x}_n, \mathbf{x}_m) = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}_m)$ là hàm kernel và $\mathbf{k}(\mathbf{x}^*)_n = k(\mathbf{x}_n, \mathbf{x}^*)$. Các kernel phổ biến:

- **Polynomial:** $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d$,
- **RBF / Gaussian:** $k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$,
- **Matérn:** phù hợp khi hàm mục tiêu kém trơn (less smooth).

2.1.10. Gaussian Process Regression [Nâng cao]

Gaussian Process (GP) là một phân phối trên các hàm số: mọi tập hữu hạn giá trị hàm có phân phối Gaussian đa biến. GP Regression:

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')). \quad (15)$$

Phân phối dự đoán tại $\mathbf{x}^*$:

$$p(f^* | \mathbf{x}^*, \mathbf{X}, \mathbf{t}) = \mathcal{N}(f^* | \bar{f}^*, \mathbb{V}[f^*]), \quad (16)$$

$$\bar{f}^* = \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{t}, \quad \mathbb{V}[f^*] = k_{**} - \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*. \quad (17)$$

GP mang lại ước lượng bất định (uncertainty quantification) tự nhiên và có thể xem là giới hạn của Bayesian Ridge Regression với vô hạn hàm cơ sở.

2.2. Yêu cầu Thực hiện – Phần 1

2.2.1. Chọn bộ dữ liệu

Yêu cầu

Sinh viên phải chọn **một bộ dữ liệu hồi quy thực tế** hoặc **tự thu thập**, thỏa mãn các điều kiện sau:

- **Số lượng mẫu và Số lượng đặc trưng:** ít nhất 10,000 mẫu và số lượng biến đầu vào ít nhất 5 biến (trong đó ít nhất 3 biến liên tục).
- **Nguồn dữ liệu:** lấy từ các kho dữ liệu công khai uy tín như UCI ML Repository (<https://archive.ics.uci.edu>), Kaggle (<https://www.kaggle.com>), hoặc các cơ sở dữ liệu khoa học.
- **Cấm:** không sử dụng dữ liệu tổng hợp (synthetic data).

2.2.2. Phân tích và Tiền xử lý Dữ liệu

Sinh viên thực hiện và báo cáo đầy đủ các bước sau:

Yêu cầu

1. **Mô tả dữ liệu:** Kích thước, kiểu dữ liệu, ý nghĩa từng biến.
2. **Phân tích khám phá (EDA):**
 - Thống kê mô tả: mean, std, min, max, tứ phân vị.
 - Phân bố của biến mục tiêu (histogram, boxplot).
 - Ma trận tương quan và biểu đồ scatter các cặp biến quan trọng.
 - Phát hiện ngoại lai (outliers) bằng phương pháp IQR hoặc Z-score. Sinh viên có thể tìm hiểu và áp dụng những phương pháp phát hiện ngoại lai khác.

3. Tiền xử lý:

- Xử lý giá trị còn thiếu (missing values): nếu có.
- Chuẩn hóa (normalization / standardization): giải thích lý do chọn.
- Chia dữ liệu: train / validation / test (ví dụ: 70% / 10% / 20%).

2.2.3. Xây dựng và Huấn luyện Mô hình

Sinh viên cần xây dựng và so sánh **ba mô hình** sau:

Yêu cầu

1. Hồi quy Tuyến tính (Linear Regression):

- Cài đặt **Normal Equations** từ đầu (không dùng `sklearn.LinearRegression`).
- Cài đặt **Mini-batch Gradient Descent** với learning rate schedule (step decay hoặc cosine annealing) và so sánh tốc độ hội tụ với Normal Equations.
- Kiểm tra giả thiết Gauss–Markov: vẽ residual plot, QQ-plot, thực hiện Breusch–Pagan test.
- Nếu phát hiện heteroscedasticity: cài đặt Weighted Least Squares (WLS).

2. Hồi quy có Regularization và Lựa chọn Đặc trưng:

- Cài đặt **Ridge Regression** (L2) và **Lasso Regression** (L1).
- Chọn λ tốt nhất bằng k -fold CV ($k = 10$) – sử dụng cả *grid search* lẫn *warm start* (Lasso path).
- Vẽ **regularization path**: hệ số w theo $\log \lambda$ cho cả Ridge và Lasso.
- Cài đặt **Elastic Net**: $E(w) = \frac{1}{2} \|t - \Phi w\|^2 + \lambda_1 \|w\|_1 + \frac{\lambda_2}{2} \|w\|^2$ và phân tích vùng tối ưu (λ_1, λ_2) .
- Thực hiện **Lựa chọn đặc trưng**: so sánh Forward Stepwise Selection, Backward Elimination, và chọn đặc trưng dựa trên Lasso (nonzero coefficients).

3. Mô hình với Hàm Cơ sở Phi Tuyến và Ablation Study:

- Áp dụng ít nhất **ba loại** hàm cơ sở (polynomial, Gaussian RBF, và một loại tự chọn).
- Vẽ **validation curve**: MSE theo bậc đa thức / số hàm cơ sở.
- Thực hiện **ablation study**: lần lượt bỏ từng nhóm đặc trưng hoặc từng loại hàm cơ sở, đo tác động đến hiệu năng – từ đó xác định đặc trưng / hàm cơ sở quan trọng nhất.
- Phân tích **hiệu ứng tương tác**: thêm các hạng $x_i x_j$ vào mô hình và đánh giá mức cải thiện.

Sinh viên có thể cài đặt thêm những mô hình khác để so sánh để làm phong phú thêm cho báo cáo.

Sinh viên có thể thực hiện thêm để nhận điểm bonus:

Nâng cao

- **Hồi quy Bayesian đầy đủ + vùng bất định:** tính posterior $p(\mathbf{w} \mid \mathbf{t})$, vẽ predictive distribution $\bar{f}^* \pm 2\sigma_N$ trên tập test.
- **Evidence Maximization:** tối ưu α, β bằng re-estimation equations (EM-style), so sánh với CV về thời gian và chất lượng.
- **Kernel Ridge Regression:** cài đặt với ít nhất hai kernel (RBF, polynomial), chọn bandwidth ℓ bằng CV, so sánh với Ridge Regression tuyến tính.
- **Gaussian Process Regression:** cài đặt GP với RBF kernel, tối ưu log-marginal-likelihood bằng gradient ascent để học tham số kernel, vẽ posterior predictive với error bars.
- **Robust Regression:** cài đặt IRLS với phân phối Student- t hoặc Huber loss, so sánh độ nhạy với outlier so với OLS.
- **Phân tích Bias–Variance thực nghiệm:** bootstrap 200 lần, ước lượng bias² và variance, vẽ cùng trục với regularization path.

[+5 điểm bonus]

2.2.4. Đánh giá Mô hình

Yêu cầu

Sinh viên sử dụng đầy đủ các chỉ số đánh giá sau và báo cáo trên tập **test**:

Chỉ số	Công thức	Ý nghĩa
MSE	$\frac{1}{N} \sum (t_n - y_n)^2$	Sai số bình phương trung bình
RMSE	$\sqrt{\text{MSE}}$	Căn bậc hai MSE
MAE	$\frac{1}{N} \sum t_n - y_n $	Sai số tuyệt đối trung bình
R^2	$1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$	Hệ số xác định

Ngoài ra, sinh viên cần:

- Vẽ biểu đồ **learning curves** (train loss và val loss theo số lượng mẫu huấn luyện).
- Vẽ biểu đồ **phần dư** (residuals): kiểm tra tính ngẫu nhiên của sai số.
- Vẽ biểu đồ **giá trị dự đoán vs. giá trị thật** (predicted vs. actual).
- So sánh tất cả các mô hình trong một bảng kết quả thống nhất.
- Thực hiện **k-fold cross-validation** ($k = 10$) để báo cáo mean $\pm$ std cho từng chỉ số.

- Kiểm định thống kê **paired t -test** hoặc **Wilcoxon signed-rank test** để xác nhận sự khác biệt có ý nghĩa thống kê giữa các mô hình.

2.2.5. Phân tích và Thảo luận

Yêu cầu

Sinh viên trình bày phân tích và thảo luận bao gồm:

- So sánh hiệu năng các mô hình: mô hình nào tốt nhất? Tại sao?
- Phân tích ảnh hưởng của regularization: λ ảnh hưởng như thế nào đến độ lệch (bias) và phương sai (variance)?
- Phân tích hiện tượng overfitting / underfitting nếu có.
- Hạn chế của các mô hình tuyến tính đối với bài toán cụ thể.
- Đề xuất hướng cải tiến hoặc mô hình phức tạp hơn.
- **Phân tích độ phức tạp tính toán:** so sánh $\mathcal{O}(\cdot)$ của Normal Equations vs. Gradient Descent vs. Bayesian Regression theo N và M .
- **Phân tích ảnh hưởng của kích thước tập huấn luyện** đến hiệu năng: vẽ learning curve và nhận xét về tốc độ hội tụ.

2.3. Phân bổ Điểm – Phần 1

Điểm số

STT	Hạng mục	Điểm
1	Cơ sở lý thuyết: Normal Eq., Gauss–Markov, Bias–Variance, Bayesian, Evidence	13
2	EDA, kiểm định Breusch–Pagan, WLS nếu cần, tiền xử lý đầy đủ	7
3	Linear Regression: Normal Equations + Mini-batch GD + learning rate schedule	8
4	Elastic Net + Lasso path + Forward/Backward feature selection	10
5	Mô hình phi tuyến (3 loại) + validation curve + ablation study	7
6	Đánh giá: k -fold CV $k = 10$, kiểm định thống kê, calibration	5
Tổng Phần 1 (cơ bản)		50
7	<i>Bonus: Hồi quy Bayesian/ Evidence Maximization/ Kernel Ridge / GP Regression / Robust Regression / Bias–Var bootstrap</i>	+5

3. Phần 2 – Bài toán Phân lớp (Classification)

Tổng quan Phần 2

Sinh viên tìm hiểu và áp dụng các **mô hình tuyến tính cho bài toán phân lớp** (Linear Models for Classification). Sinh viên **nhận** tham khảo **Chương 4** của sách *Pattern Recognition and Machine Learning* (Bishop, 2006).

Tổng điểm Phần 2: **[50 điểm]**

3.1. Cơ sở Lý thuyết

3.1.1. Bài toán Phân lớp

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^N$ trong đó $\mathbf{x}_n \in \mathbb{R}^D$ và $c_n \in \{1, 2, \dots, K\}$ là nhãn lớp. Mục tiêu là xây dựng hàm phân tách $f : \mathbb{R}^D \rightarrow \{1, \dots, K\}$ sao cho tỷ lệ phân lớp sai trên dữ liệu mới là nhỏ nhất.

3.1.2. Các chiến lược phân lớp

Định nghĩa 3.1 (Phân loại các mô hình phân lớp tuyến tính). Có ba chiến lược chính:

1. **Discriminant Functions:** Học trực tiếp hàm phân tách $f(\mathbf{x})$ mà không mô hình xác suất.
2. **Probabilistic Discriminative Models:** Mô hình hóa trực tiếp $p(c | \mathbf{x})$ (ví dụ: Logistic Regression).
3. **Probabilistic Generative Models:** Mô hình hóa $p(\mathbf{x} | c)$ và $p(c)$, rồi sau đó áp dụng định lý Bayes.

3.1.3. Hàm phân quyết tuyến tính

Với bài toán phân lớp nhị phân ($K = 2$):

$$y(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0. \quad (18)$$

Mặt phẳng quyết định (decision surface) là tập hợp $\{\mathbf{x} : y(\mathbf{x}) = 0\}$, là một siêu phẳng (hyperplane) trong $\mathbb{R}^D$.

Với bài toán đa lớp ($K > 2$), kiến trúc **one-versus-the-rest** hoặc **one-versus-one** có thể sử dụng, hoặc mô hình K -hàm phân quyết:

$$y_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + w_{k0}, \quad k = 1, \dots, K. \quad (19)$$

Điểm $\mathbf{x}$ được gán vào lớp $k^* = \arg \max_k y_k(\mathbf{x})$.

3.1.4. Phân tích phân tách tuyến tính Fisher (Fisher's LDA)

Fisher's Linear Discriminant Analysis (LDA) tìm hướng chiếu w sao cho với hàm năng lượng được định nghĩa:

$$J(w) = \frac{w^\top S_B w}{w^\top S_W w} \quad (20)$$

được tối đa hóa, trong đó:

$$S_B = (m_2 - m_1)(m_2 - m_1)^\top, \quad (21)$$

$$S_W = \sum_{n \in \mathcal{C}_1} (x_n - m_1)(x_n - m_1)^\top + \sum_{n \in \mathcal{C}_2} (x_n - m_2)(x_n - m_2)^\top. \quad (22)$$

Nhiệm tối ưu của bài toán (đã được chứng minh): $w \propto S_W^{-1}(m_2 - m_1)$.

3.1.5. Hồi quy Logistic (Logistic Regression)

Mô hình **Logistic Regression** là mô hình phân lớp xác suất phân biệt (discriminative) quan trọng nhất trong lớp các mô hình tuyến tính.

Với bài toán phân lớp nhị phân:

$$p(\mathcal{C}_1 | x) = \sigma(w^\top x + w_0) = \frac{1}{1 + e^{-(w^\top x + w_0)}}, \quad (23)$$

trong đó $\sigma(\cdot)$ là hàm sigmoid.

Với bài toán đa lớp (Softmax Regression):

$$p(\mathcal{C}_k | x) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \quad a_k = w_k^\top x + w_{k0}. \quad (24)$$

Hàm mất mát **cross-entropy**:

$$E(W) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln p(\mathcal{C}_k | x_n). \quad (25)$$

Gradient đối với w_k :

$$\nabla_{w_k} E = \sum_{n=1}^N (y_{nk} - t_{nk}) x_n, \quad y_{nk} = p(\mathcal{C}_k | x_n). \quad (26)$$

3.1.6. Mô hình sinh Gaussian (Generative LDA)

Mô hình sinh giả thiết mỗi lớp tuân theo phân phối Gaussian với cùng ma trận hiệp phương sai được viết như sau:

$$p(x | \mathcal{C}_k) = \mathcal{N}(x | \mu_k, \Sigma). \quad (27)$$

Ước lượng tham số bằng phương pháp Maximum Likelihood, ta được:

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{n \in \mathcal{C}_k} x_n, \quad \hat{\Sigma} = \sum_{k=1}^K \frac{N_k}{N} S_k. \quad (28)$$

3.1.7. Xấp xỉ Laplace [Nâng cao]

Xấp xỉ Laplace ước lượng posterior của Logistic Regression bằng một phân phối Gaussian tâm tại MAP estimate $\mathbf{w}_{\text{MAP}}$ được viết như sau:

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{w}_{\text{MAP}}, \mathbf{A}^{-1}), \quad (29)$$

trong đó $\mathbf{A} = -\nabla \nabla \ln p(\mathbf{w} \mid \mathcal{D})|_{\mathbf{w}_{\text{MAP}}}$ là ma trận Hessian âm của log-posterior. Xấp xỉ này cho phép lượng hóa bất định của dự đoán, khác với Logistic Regression thuần túy chỉ cho điểm ước lượng.

3.1.8. Hàm Probit [Nâng cao]

Thay vì hàm sigmoid, mô hình Probit sử dụng hàm phân phối tích lũy của Gaussian chuẩn như sau:

$$p(\mathcal{C}_1 \mid \mathbf{x}) = \Phi(\mathbf{w}^\top \mathbf{x} + w_0), \quad \Phi(a) = \int_{-\infty}^a \mathcal{N}(t \mid 0, 1) dt. \quad (30)$$

Mô hình Probit phát sinh tự nhiên từ mô hình biến ẩn (latent variable model) và phù hợp hơn khi dữ liệu nhãn bị nhiễu (noisy labels).

3.1.9. Thuật toán Newton–Raphson / IRLS [Nâng cao]

Logistic Regression không có nghiệm dạng đóng, nhưng hàm mất mát cross-entropy là **lồi** và có Hessian xác định dương. Thuật toán Newton–Raphson (hay IRLS – Iterative Reweighted Least Squares) sử dụng thông tin bậc hai, biểu thức cập nhật có dạng như sau:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \mathbf{H}^{-1} \nabla E, \quad (31)$$

trong đó Hessian:

$$\mathbf{H} = \Phi^\top \mathbf{R} \Phi + \lambda \mathbf{I}, \quad \mathbf{R} = \text{diag}(y_{n1}(1 - y_{n1}), \dots, y_{nN}(1 - y_{nN})), \quad (32)$$

IRLS hội tụ trong $\mathcal{O}(\log(1/\epsilon))$ bước – nhanh hơn nhiều so với Gradient Descent cần $\mathcal{O}(1/\epsilon)$ bước. Tuy nhiên chi phí mỗi bước là $\mathcal{O}(NM^2 + M^3)$ do nghịch đảo Hessian.

3.1.10. Thuật toán Perceptron

Thuật toán Perceptron (Rosenblatt, 1958) là một trong những thuật toán học máy lâu đời nhất:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla E_P(\mathbf{w}), \quad (33)$$

với hàm mất mát Perceptron:

$$E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^\top \phi(\mathbf{x}_n) t_n, \quad (34)$$

trong đó $\mathcal{M}$ là tập các mẫu bị phân lớp sai. **Định lý hội tụ Perceptron** đảm bảo thuật toán hội tụ trong hữu hạn bước *nếu và chỉ nếu* dữ liệu có thể phân tách tuyến tính. Thuật toán học Perceptron là tiền thân của các mạng nơ-ron sâu hiện đại.

3.1.11. Kernel Logistic Regression [Nâng cao]

Tương tự Kernel Ridge Regression, ta có thể kernelize Logistic Regression bằng cách biểu diễn:

$$y(\mathbf{x}) = \sigma \left(\sum_{n=1}^N a_n k(\mathbf{x}_n, \mathbf{x}) \right), \quad (35)$$

trong đó $\mathbf{a} = (a_1, \dots, a_N)^\top$ là các hệ số dual. Cập nhật dual:

$$\nabla_{\mathbf{a}} E = \mathbf{K}(\mathbf{y} - \mathbf{t}) + \lambda \mathbf{K} \mathbf{a}, \quad (36)$$

trong đó $\mathbf{K}$ là **ma trận Gram**. Kernel Logistic Regression cho phép học đường biên quyết định phi tuyến mà vẫn giữ diễn giải xác suất.

3.1.12. Thuật toán tối ưu hóa

Thông thường, chúng ta hay sử dụng phương pháp hướng giảm (descent method) để tối ưu. Tùy vào thông tin cục bộ mà thuật toán sử dụng mà ta có thể phân loại nó thành hai nhóm chính: các thuật toán bậc nhất (first-order) và bậc hai (second-order). Tham khảo sách: [Algorithm for Optimization](#) của Mykel J. Kochenderfer và Tim A. Wheeler.

Thuật toán 3.1 (Gradient Descent cho Logistic Regression). 1. Khởi tạo $\mathbf{w} = \mathbf{0}$ hoặc ngẫu nhiên nhỏ.

2. Lặp lại đến khi hội tụ:

- (a) Tính $\hat{y}_n = \sigma(\mathbf{w}^\top \mathbf{x}_n)$ cho mọi n .
- (b) Tính gradient: $\nabla E = \sum_n (\hat{y}_n - t_n) \mathbf{x}_n$.
- (c) Cập nhật: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$.

3. Trả về $\mathbf{w}$.

3.2. Yêu cầu Thực hiện – Phần 2

3.2.1. Chọn bộ dữ liệu

Yêu cầu

Sinh viên phải chọn **một bộ dữ liệu phân lớp thực tế** hoặc **tự thu thập**, thỏa mãn các tiêu chí sau:

- **Số lượng mẫu và Số lượng đặc trưng:** ít nhất 10,000 mẫu và số lượng biến đầu vào ít nhất 5 biến.
- **Số lượng lớp:** phân lớp nhị phân (2 lớp) hoặc đa lớp ($K \geq 3$).
- **Nguồn:** UCI ML Repository, Kaggle, hoặc tương đương.
- **Cấm:** không sử dụng dữ liệu tổng hợp.

3.2.2. Xây dựng và Huấn luyện Mô hình

Sinh viên cần xây dựng và so sánh **ba mô hình** sau:

Yêu cầu

1. Logistic Regression (nhị phân và đa lớp):

- Cài đặt **Gradient Descent** từ đầu cho nhị phân (sigmoid) và đa lớp (softmax).
- Cài đặt **Newton–Raphson** / **IRLS** cho bài toán nhị phân và so sánh tốc độ hội tụ (loss vs. epoch, loss vs. wall-clock time).
- Cho bài toán đa lớp: so sánh ba chiến lược **one-vs-rest**, **one-vs-one**, và **softmax trực tiếp**.
- Trình bày đầy đủ đạo hàm Hessian, Jacobian của softmax.

2. Linear Discriminant Analysis – LDA và QDA:

- Cài đặt **LDA** (hiệp phương sai chung $\hat{\Sigma}$) và **QDA** (hiệp phương sai riêng $\hat{\Sigma}_k$).
- Tính Fisher ratio $J(\mathbf{w})$ cho từng đặc trưng, xếp hạng độ phân biệt lớp.
- Vẽ quy chiếu LDA xuống 2D và biểu diễn **đường biên quyết định (decision boundary)**.
- Phân tích khi nào LDA tốt hơn QDA và ngược lại (liên hệ với số mẫu và số chiều).

3. Perceptron và Logistic Regression có Regularization:

- Cài đặt **Perceptron algorithm** từ đầu, quan sát hội tụ trên dữ liệu phân chia tuyến tính và không phân chia tuyến tính.
- Áp dụng **L1 và L2 regularization** cho Logistic Regression, so sánh sparsity và hiệu năng.
- Chọn λ bằng **stratified k -fold CV** ($k = 5$) để giữ tỷ lệ lớp trong mỗi fold.
- Cài đặt **class-weighted loss** để xử lý mất cân bằng: $E = - \sum_{n,k} c_k t_{nk} \ln y_{nk}$ với $c_k \propto N/N_k$.

Sinh viên có thể cài đặt thêm những mô hình khác để so sánh để làm phong phú thêm cho báo cáo.

Nâng cao

Sinh viên có thể thực hiện thêm để nhận điểm bonus:

- **Mô hình Probit:** cài đặt và so sánh với Logistic Regression – đường biên, xác suất đầu ra, và độ nhạy với nhiễu.
- **Xấp xỉ Laplace:** tính Hessian tại MAP, xây dựng posterior Gaussian, vẽ vùng bất định $\pm 2\sigma$ của đường biên quyết định.
- **Kernel Logistic Regression:** cài đặt dual formulation với RBF kernel, so sánh với

mô hình tuyến tính trên dữ liệu không phân chia tuyến tính (XOR-like patterns).

- **Gaussian Naive Bayes vs. LDA:** so sánh lý thuyết (GNB giả thiết independence, LDA không) và thực nghiệm – khi nào GNB thực tế tốt hơn LDA dù giả thiết kém chặt hơn?
- **Phân tích VC dimension:** tính VC dimension của lớp phân lớp tuyến tính trong $\mathbb{R}^D$ và liên hệ với tổng quát hóa thông qua structural risk minimization.

[+5 điểm bonus]

3.2.3. Đánh giá Mô hình

Yêu cầu

Sinh viên báo cáo đầy đủ các chỉ số đánh giá trên tập **test**:

Chỉ số	Mô tả
Accuracy	Tỷ lệ mẫu phân lớp đúng trên tổng số mẫu
Precision	Độ chính xác: $TP / (TP + FP)$ (mỗi lớp)
Recall	Độ phủ: $TP / (TP + FN)$ (mỗi lớp)
F1-score	Trung bình điều hòa của Precision và Recall
Confusion Matrix	Ma trận nhầm lẫn (vẽ biểu đồ nhiệt)
ROC curve + AUC	Vẽ trên tập test (cho bài phân lớp nhị phân)

Ngoài ra:

- Vẽ đồ thị hàm mất mát (cross-entropy loss) theo số vòng lặp (epochs).
- Vẽ đường biên giới quyết định (decision boundary) trong không gian 2D nếu có thể.
- So sánh tất cả các mô hình trong một bảng kết quả thống nhất.
- Thực hiện **k -fold cross-validation** ($k = 5$) để báo cáo mean $\pm$ std cho mỗi chỉ số.
- Vẽ **Precision–Recall curve** và so sánh Average Precision (AP) giữa các mô hình.
- Thực hiện **McNemar’s test** để kiểm định xem hai mô hình có sự khác biệt thống kê trên cùng tập test hay không.
- Phân tích **calibration** của xác suất đầu ra bằng reliability diagram (biểu đồ độ tin cậy).

3.2.4. Phân tích và Thảo luận

Yêu cầu

Sinh viên trình bày phân tích:

- So sánh hiệu năng ba mô hình: ưu và nhược điểm của mỗi mô hình.
- Logistic Regression vs. LDA: khi nào nên dùng mô hình phân biệt, khi nào nên dùng mô hình sinh?
- Phân tích hiệu quả của regularization đối với hiệu suất phân lớp của mô hình.
- Nếu dữ liệu mất cân bằng: các chỉ số nào phản ánh đúng chất lượng mô hình hơn?
- Hạn chế của mô hình tuyến tính khi dữ liệu không phân chia tuyến tính.
- **Phân tích lỗi:** xem xét các mẫu bị phân lớp sai, có điểm chung nào không?
- **Phân tích discriminability:** với LDA, tính tỷ số Fisher $J(\mathbf{w})$ cho từng đặc trưng riêng lẻ và xếp hạng mức độ phân biệt lớp.
- **So sánh với phương pháp phi tuyến (nếu có):** nhận xét về khoảng cách hiệu năng và chi phí giải thích (interpretability).

3.3. Phân bố Điểm – Phần 2

Điểm số

STT	Hạng mục	Điểm
1	Lý thuyết: gradient/Hessian LR, IRLS, Perceptron, Kernel LR, Laplace, Probit	13
2	EDA, phân tích mất cân bằng, tiền xử lý có chiều sâu	6
3	Logistic Regression: GD + Newton–Raphson + so sánh 3 chiến lược đa lớp	10
4	LDA + QDA: Fisher ratio, so sánh lý thuyết, quy chiếu 2D	9
5	Perceptron + L1/L2 LR + class-weighted loss + stratified CV	8
6	Đánh giá: CV, McNemar, PR curve, calibration, phân tích lỗi sâu	4
Tổng Phần 2 (cơ bản)		50
7	<i>Bonus: Probit / Laplace / Kernel LR / GNB vs LDA / VC dimension</i>	<i>+5</i>

4. Phần 3 – Phân tích So sánh và Kỹ năng Nghiên cứu

Tổng quan Phần 3 (không tính điểm riêng – tích hợp vào báo cáo)

Phần này không có điểm độc lập mà được đánh giá như một **tiêu chí chất lượng bắt buộc** của phần viết báo cáo. Sinh viên nên thể hiện khả năng tư duy phản biện, tổng hợp và kết nối kiến thức giữa hai phần.

4.1. Kết nối Hồi quy và Phân lớp

Yêu cầu

Sinh viên nên thảo luận về mối quan hệ sâu giữa hai bài toán:

- **Hồi quy logistic như hồi quy:** Logistic Regression thực chất là hồi quy hàm sigmoid – mô tả mối quan hệ nào giữa hai bài toán?
- **Generalized Linear Models (GLM):** cả Linear Regression và Logistic Regression đều là trường hợp đặc biệt của GLM với link function khác nhau. Trình bày khung thống nhất này.
- **Exponential family:** Gaussian (hồi quy) và Bernoulli/Categorical (phân lớp) đều thuộc exponential family – điều này tạo ra cấu trúc toán học thống nhất như thế nào?

4.2. Bảng so sánh toàn diện

Yêu cầu

Sinh viên nên lập bảng so sánh **tất cả các mô hình** đã cài đặt trong cả hai phần, bao gồm:

Tiêu chí	Linear Reg.	Ridge/Lasso	Logistic Reg.	LDA/QDA
Giả thiết phân phối				
Nghiệm dạng đóng				
Độ phức tạp huấn luyện				
Khả năng giải thích				
Nhạy với outlier				
Hiệu năng thực nghiệm				

4.3. Phân tích mức độ nhạy cảm (sensitivity) và tính bền vững (robustness)

Yêu cầu

- **Sensitivity analysis:** thay đổi tỷ lệ train/test split (60/40, 70/30, 80/20), vẽ boxplot hiệu năng – mô hình nào ổn định nhất?
- **Noise injection:** thêm Gaussian noise với $\sigma \in \{0.1, 0.5, 1.0\}$ vào đặc trưng đầu vào, quan sát mức suy giảm hiệu suất – mô hình nào *bền vững (robust)* hơn? (Hay khả năng chống chịu nhiễu loạn của mô hình nào tốt hơn?)
- **Feature corruption:** xóa ngẫu nhiên 10%, 20%, 30% giá trị đặc trưng, so sánh chiến lược imputation khác nhau (mean, median, KNN imputation).
- **Phân tích khả năng hội tụ (convergence):** vẽ loss curve cho tất cả các thuật toán lặp, xác định số epochs cần thiết để đạt được hiệu suất tốt nhất có thể.

4.4. Khả năng tái hiện lại thí nghiệm

Yêu cầu

- Cố định tất cả random seed.
- Ghi log đầy đủ: hyperparameter, train/val/test split indices, metric từng fold, ...
- Báo cáo **thời gian thực thi/ huấn luyện/ kiểm tra** và **bộ nhớ tiêu tốn** của từng thuật toán.
- Cung cấp file `environment.yml` (conda) hoặc `requirements.txt` với version cụ thể.
- Kết quả phải tái hiện được hoàn toàn khi chạy lại notebook từ đầu.

5. Tổng kết Điểm số

Phần	Nội dung	Điểm tối đa	Hệ số
1	Bài toán Hồi quy (Linear Models for Regression)	50	50%
2	Bài toán Phân lớp (Linear Models for Classification)	50	50%
Tổng điểm cơ bản		100	100%
<i>Bonus Phần 1 (Kernel Regression / GP / Robust / Bias–Var)</i>		+5	—
<i>Bonus Phần 2 (Probit / Laplace / Kernel LR / GNB / VC dim)</i>		+5	—
Tổng tối đa (kể cả bonus)		110	—

! Lưu ý

Tiêu chí đánh giá chất lượng báo cáo (áp dụng cho cả hai phần):

- Trình bày rõ ràng, mạch lạc, có cấu trúc logic.
- Công thức toán học chính xác, có giải thích ý nghĩa các ký hiệu, tính thống nhất trên toàn báo cáo.
- Biểu đồ đẹp, có tên trục, chú thích, tiêu đề.
- Code sạch đẹp, có comment, có thể chạy lại/ thực nghiệm được.
- Phân tích kết quả sâu sắc, thể hiện hiểu biết của nhóm sinh viên về mô hình/ thuật toán.
- Trích dẫn tài liệu đúng chuẩn.

6. Quy định Nộp bài

6.1. Cấu trúc thư mục nộp

Cấu trúc thư mục

```
Group_ID/
|-- report/
|   |-- report.pdf
|   |-- report.tex
|-- code/
|   |-- Part1_Regression/
|   |   |-- notebook.ipynb
|   |   |-- utils.py
|   |-- Part2_Classification/
|   |   |-- notebook.ipynb
|   |   |-- utils.py
|-- data/
|   |-- README.md
|-- README.md
```

6.2. Yêu cầu về Code

Yêu cầu

- Tất cả code phải chạy lại được.
- Phần cài đặt từ đầu phải rõ ràng, tách biệt với phần sử dụng thư viện kiểm chứng.
- Phiên bản Python và thư viện phải được ghi rõ trong file `requirements.txt`.
- Đặt seed ngẫu nhiên để đảm bảo kết quả tái hiện được.

6.3. Yêu cầu về Báo cáo

Yêu cầu

- Báo cáo được nộp dưới dạng **PDF**, không giới hạn độ dài trang (không tính phụ lục, bìa, mục lục, lời cảm ơn). Danh mục tài liệu tham khảo theo chuẩn **IEEE** hoặc **APA** rõ ràng.
- Khuyến nghị sử dụng $\text{\LaTeX}$. Nếu sử dụng $\text{\LaTeX}$, yêu cầu nhóm nộp cả mã nguồn `.tex`.
- Mỗi thành viên trong nhóm phải có mục **đóng góp rõ ràng**.

6.4. Hình thức và Hạn nộp

Hình thức nộp: Moodle

Định dạng: File `.zip` đặt tên theo `Group_<ID>.zip`

Trễ hạn: Trừ 10% điểm cho mỗi ngày trễ. Sau 3 ngày: 0 điểm.

Quy định khác Trừ 10% cho nộp bài sai quy định.

Nếu kích thước bài nộp lớn hơn 25MB, sinh viên upload đồ án lên Google Drive/ One Drive và chia sẻ link trong một file duy nhất `Group_ID.txt`. Nếu sinh viên quản lý đồ án bằng các công cụ version control như Github thì sinh viên cũng có thể nộp link repository Github.

! Lưu ý

Lưu ý về học thuật:

- Mỗi sinh viên/nhóm phải tự thực hiện đồ án. **Nghiêm cấm sao chép bài** của nhóm khác.
- Việc sử dụng AI hỗ trợ (ChatGPT, Copilot, v.v.) được phép **để hiểu lý thuyết và debug**, nhưng sinh viên phải hiểu rõ và có thể giải thích toàn bộ code và nội dung báo cáo.

- Nếu phát hiện gian lận, tất cả thành viên liên quan sẽ bị **0 điểm toàn bộ đồ án** và xử lý theo quy định Nhà trường.

Tài liệu

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chương 3 (*Linear Models for Regression*) và Chương 4 (*Linear Models for Classification*) là tài liệu tham khảo chính của đồ án này.
- [2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- [3] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
- [4] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org>
- [6] Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. <https://archive.ics.uci.edu>